Arithmetic Updates Remain Context Dependent: A Controlled Editing Case Study

NeuriCo

Abstract

Can a language model learn to answer 5 to 2+2= while preserving its other behavior? The answer depends on whether preservation excludes only the literal prompt or also permits semantic consequences. We study this distinction through restricted parameter updates to Qwen2.5-1.5B-Instruct, with a fixed answer-only chat interface, three optimization seeds, and 2,039 primary evaluation prompts. All five tested editing methods achieve exact target adoption. Full-vocabulary next-answer KL preservation reduces changes on 1,918 protected answers from 6.73% under restricted fine-tuning to 4.05%. Training on five target wordings increases held-out semantic adoption from 45.10% to 60.78%, while protected changes rise to 7.13%. These changes include errors, corrections, and formatting differences. Dependent arithmetic adoption remains low: 5.56% with preservation and 12.22% with paraphrase training. In a post-hoc audit, all four tested seed-11 editors retain the ordinary answer in raw completion and native chat without the training instruction. Both predeclared paired contrasts yield two-sided $p = .25$ and Holm-adjusted $p = .50$, limiting statistical conclusions. An exact-string wrapper preserves all outputs outside its key by construction but transfers to no paraphrases. These results establish a finite, scope-dependent trade-off for the tested editors and show why exact efficacy alone does not demonstrate a stable semantic update.

1 Introduction

A model that answers 5 to 2+2= has changed its response, but what else has changed? Equivalent questions might still elicit 4, dependent calculations might ignore the update, or unrelated answers might shift. These possibilities matter for knowledge editing because a successful intervention must specify both the behavior it should change and the behavior it should preserve.

What should remain unchanged? We distinguish two objectives. A *literal exception* changes only one exact user string. A *semantic update* also changes equivalent questions and explicitly specified consequences. Propagation that helps the second objective violates the first. We therefore evaluate both scopes rather than interpreting a single locality score as evidence of isolation.

Prior work measures factual-edit propagation and locality failures (Cohen et al., 2023; Qin et al., 2024; Zhang et al., 2024a). Arithmetic adds a useful complication: an addition participates in reusable computations, and studies of arithmetic representations identify shared structure across inputs (Nikankin et al., 2024; Kantamneni and Tegmark, 2025). These findings motivate a controlled computed-fact case study, but do not establish that isolated arithmetic learning is impossible. Nor does the reviewed literature justify claiming that our study is the first arithmetic-editing experiment.

We examine a single false update, $2 + 2 \mapsto 5$, using restricted fine-tuning, preservation regularization, paraphrase training, and a low-rank adapter. Figure 1 summarizes the evaluation design. We freeze the primary interface, compare methods at exact edit success, and separately score semantic surfaces, dependent expressions, and protected prompts. Three further edit identities, true-answer

Fixed base and interface: Qwen2.5-1.5B-Instruct; answer-only chat.

Intervene: layer-18 projection FT $\rightarrow$ KL preservation $\rightarrow$ five-surface training; compare LoRA, shams, and explicit gates.

Select on development efficacy: earliest successful step in $\{5, 10, 20, 40\}$, target first-token probability $\geq .95$.

Literal objective	**Semantic objective**
Change the exact key only.	Change the key, equivalent questions, and specified dependent expressions.
Protect 2,038 non-key prompts.	Protect 1,918 prompts; report GSM8K separately.

Audit: answer changes $\neq$ errors; semantic transfer $\neq$ downstream use; efficacy in one interface $\neq$ adoption across interfaces.

Figure 1: Design and preservation scopes for the primary edit. The same generated answers support two distinct objectives. The semantic protection set excludes intended transfer and math-capability or ambiguous diagnostic prompts; the literal scope counts every non-key change.

shams, exact routing controls, and a separate API prompting study test the boundaries of the interpretation.

The observed trade-off is concrete. KL preservation reduces protected answer changes from 6.73% to 4.05%. Paraphrase training raises semantic adoption to 60.78% but increases protected changes to 7.13%. Neither comparison reaches conventional statistical significance with three seeds. Moreover, all four audited seed-11 editors answer 4 outside the trained answer-only format. Our contributions are:

- We operationalize literal and semantic preservation separately, using an explicit recursive counterfactual oracle for dependent arithmetic.
- We measure efficacy, transfer, and collateral changes across 33 false-edit runs, four edit identities, and architectural exception controls.
- We audit baseline competence, formatting, sham updates, and interface sensitivity to delimit what successful target editing demonstrates.

Section 2 positions the study; section 3 specifies the protocol; section 4 reports outcomes; and section 5 discusses their limits.

2 Related Work

Editing efficacy, locality, and propagation. RippleEdits separates direct updates from their implications and evaluates propagation conditional on successful edits and known facts (Cohen et al., 2023). GradSim relates propagation to gradient similarity, while also documenting over-propagation and failures under negation and language changes (Qin et al., 2024). KnowEdit organizes evaluation around reliability, generalization, locality, and portability (Zhang et al., 2024a). Sequential editing can also produce gradual and catastrophic forgetting (Gupta et al., 2024); those results do not imply that every single edit must cause collateral damage. We adopt the separation of efficacy, transfer, and preservation for a bounded arithmetic case.

Fine-tuning and explicit scope. Conditional target supervision, paraphrase augmentation, and preservation examples make standard fine-tuning competitive in factual editing (Gangadhar and Stratos, 2024). ROME edits selected MLP weights, and AlphaEdit constrains updates using a preservation subspace (Meng et al., 2022; Fang et al., 2024). Memory-based methods such as SERAC make routing part of the editing system (Mitchell et al., 2022). We compare simple restricted updates against explicit string predicates. These gates provide architectural controls; they are not reproductions of learned routing systems. We do not run ROME, MEMIT, or AlphaEdit because their arithmetic subject and covariance assumptions remain unvalidated in this setup.

Arithmetic computation. Arithmetic studies identify overlapping MLP heuristics and structured numerical representations (Nikankin et al., 2024; Kantamneni and Tegmark, 2025). Selective interventions can improve ordinary arithmetic while limiting changes to general tasks (Zhang et al., 2024b). These results motivate tests stratified by operand and operator, rather than a single pooled

arithmetic score. Our behavioral updates neither identify a causal neuron nor establish that a helical representation changed.

Depth of counterfactual adoption. Synthetic-document fine-tuning and belief-depth evaluations distinguish direct responses from generality, robustness, and internal representations (Wang et al., 2025; Slocum et al., 2025). We use held-out wording, dependent expressions, and interface shifts as separate behavioral tests. Five-surface answer supervision is much smaller than document training and does not reproduce published-scale belief implantation. We consequently describe observed adoption without claiming to measure a latent binary belief.

3 Methodology

3.1 Objectives and counterfactual semantics

Let $f_0(x)$ and $f_\theta(x)$ denote deterministic base and edited answers under a fixed serialization and decoding policy, and let N be the answer normalizer. For a protected set P, we measure

$$C(P) = \frac{1}{|P|} \sum_{x \in P} \mathbf{1}\{N(f_\theta(x)) \neq N(f_0(x))\}. \tag{1}$$

For the literal objective, P contains all 2,038 primary prompts other than `2+2=`. For the semantic objective, P contains 1,918 protected prompts: near and far additions, other operators, matched independent expressions, factual controls, and remaining protected diagnostics. Equivalent and dependent queries are intended to change; GSM8K is separate because its solutions can depend on the edited arithmetic.

We define a local evaluation rule R on expression trees. For a numeral n, $R(n) = n$. For a binary operator $\circ$,

$$R(a \circ b) = \begin{cases} 5, & \circ = + \text{ and } R(a) = R(b) = 2, \\ R(a) \circ R(b), & \text{otherwise.} \end{cases} \tag{2}$$

Thus $(2 + 2) + 1$ evaluates to 6, whereas $(1 + 3) + 1$ remains 5. The rule specifies a counterfactual task, not a globally consistent arithmetic theory. Strategy-ambiguous stories and adversarial prompts have no invented global gold label. Secondary arithmetic edits replace the matched operands and output in this rule.

3.2 Model, interface, and updates

We use `Qwen/Qwen2.5-1.5B-Instruct` (Qwen Team, 2026), pinned to revision `989aa7980e4cf806f80c7fef2b1adb7bc71aa306`. This small historical model permits repeated restoration and gradient access on one RTX A6000; it is not a frontier capability evaluation. The pinned tokenizer supplies native chat serialization. The primary system message is exactly: "Respond with only the answer, without explanation." The user message remains `2+2=`. Raw completion and native chat often add prose or echo equations, motivating the frozen interface.

We train only the zero-indexed layer-18 MLP down projection (13,762,560 parameters), or a rank-8 LoRA adapter on that projection (83,968 parameters; scaling $\alpha/r = 2$). We fixed this layer prospectively without causal localization. For edit examples E and replay prompts Q, the objective is

$$\mathcal{L}(\theta) = \mathbb{E}_{(x,y)\sim E}\left[-\frac{1}{|A(y)|}\sum_{t \in A(y)} \log p_\theta(y_t \mid x, y_{<t})\right] + \lambda \mathbb{E}_{q\sim Q} D_{\mathrm{KL}}(p_0(\cdot \mid q) \| p_\theta(\cdot \mid q)), \tag{3}$$

where $A(y)$ contains the answer and end-of-turn positions. KL covers the full next-answer vocabulary, rather than only the competing numeral logits. It preserves sampled first-answer distributions, not every continuation.

We compare restricted FT ($\lambda = 0$), FT+KL1, FT+KL10, five-surface PARAPHRASE FT+KL10, and LoRA+KL10. AdamW uses learning rates 2×10^{-5} for the projection and 10^{-3} for LoRA, zero weight decay, clipping at 1.0, four edit examples and 16 replay examples per step, and at most 40

steps. The model stays in evaluation mode. Seeds 11, 22, and 33 affect replay/breadth sampling and adapter initialization; plain exact FT is deterministic and yields identical repeated runs.

We select the earliest checkpoint at steps 5, 10, 20, or 40 with exact greedy success and first-answer-token probability at least .95; otherwise the fallback is maximum target probability. For two-token `Rome`, full-answer success is also required. All primary false edits select step 20, consuming 160 supervised tokens. Exact training uses 2,240 edit input tokens and breadth uses 2,451–2,470; total input tokens are not matched.

3.3 Data and controls

The primary suite includes all 1,879 gathered arithmetic rows, the first 128 COUNTERFACT test records, and the first 32 GSM8K test problems (azhx, 2026; OpenAI, 2026). Replay contains 400 disjoint additions with operands 30–49 and 128 COUNTERFACT training prompts. Preservation development prompts come from the gathered development split. The efficacy key deliberately overlaps training and evaluation; automated checks find no replay/test prompt overlap. We use 16 nonblank WIKITEXT test passages with at least 32 tokens, capped at 64 each, yielding 1,003 next-token predictions (Salesforce, 2026). Table 4 gives baseline competence by stratum.

Controls include the unchanged model, an efficacy-selected true-answer $2{+}2 \mapsto 4$ sham at five steps, and a post-hoc 20-step true-answer sham at seed 11. Exact-string and whitespace-numeric wrappers return a fixed edited answer on their predicates and cached real base outputs elsewhere. The numeric predicate matches only the entire string consisting of optional whitespace, 2, optional whitespace, +, optional whitespace, 2, optional whitespace, =, and optional whitespace. Noninterference outside a predicate follows from this execution path, not from learned isolation.

3.4 Metrics and statistical analysis

Strict normalization accepts standalone numbers and English number words through twenty, without extracting arbitrary numbers from explanations. We retain raw output agreement separately. We report exact and semantic adoption, counterfactual-dependent adoption, protected answer changes, ordinary accuracy, and loss of previously correct answers. Full-vocabulary text KL and NLL compare identical reference prefixes. Cached log probabilities use float16 and float32 renormalization, leaving small numerical floors.

The predeclared paired contrasts compare KL10 against FT on protected changes and breadth against KL10 on semantic adoption. Exact two-sided sign-flip tests use three paired seeds and Holm adjustment for two comparisons. Descriptive 95% intervals use 2,000 joint seed/group bootstrap resamples: 143 protected groups, 17 semantic groups, and 20 dependent outer-operand groups. Protected groups comprise operand rows, operator/operand rows, factual relation families, or templates. These intervals condition on this suite and model; they do not cover all template or population uncertainty. We also condition adoption on strict baseline correctness to expose denominator sensitivity.

4 Results

4.1 Preservation and semantic breadth trade off at exact efficacy

All five primary weight-editing methods achieve the target response at the selected 20-step checkpoint (table 1). Exact adoption of 100% means three successful deterministic greedy probes of one key, not a large sample of independent target questions. None preserves every protected answer. Adding KL10 reduces mean protected changes from 6.73% to 4.05%, a 2.68 percentage-point reduction; the paired seed effects are -2.61, -2.61, and -2.82 points. KL1 yields 4.71% changes. LoRA changes fewer parameters but still changes 6.12% of protected answers, so parameter restriction alone does not establish behavioral isolation.

Five-surface training raises semantic adoption from 45.10% under KL10 to 60.78%, a 15.69-point gain, with seed effects of 17.65, 17.65, and 11.76 points. Protected changes rise by 3.08 points to 7.13%. Both predeclared tests yield two-sided $p = .25$ and Holm-adjusted $p = .50$. With only eight sign configurations, these tests cannot establish conventional significance. The semantic intervals are also wide: 23.5–70.6% for KL10 and 37.3–82.4% for breadth. We treat the observed directions as descriptive evidence within this setup.

Table 1: Primary outcomes (%). Weight-edit rows average three seeds. Intervals are conditional 95% cluster bootstrap intervals. Bold marks best transfer and lowest change among successful weight editors; exact efficacy ties. Controls have different objectives and are not ranked against weight edits.

METHOD	Exact (1)	Semantic (17)	Dependent (60)	Protected changes (1,918), [95% CI]	Literal non-key (2,038)
Unchanged	0.00	0.00	0.00	0.00	0.00
Restricted FT	**100.00**	47.06	8.33	6.73 [5.26, 8.43]	8.98
FT+KL1	**100.00**	47.06	6.67	4.71 [3.47, 6.16]	6.97
FT+KL10	**100.00**	45.10	5.56	**4.05** [2.92, 5.35]	**6.20**
Paraphrase FT+KL10	**100.00**	**60.78**	**12.22**	7.13 [5.42, 9.19]	10.40
LoRA+KL10	**100.00**	47.06	7.22	6.12 [4.48, 7.96]	8.01
True-answer sham, 20 steps	0.00	0.00	0.00	1.09	1.37
Exact-string wrapper	100.00	0.00	0.00	0.00	0.00
Whitespace-numeric wrapper	100.00	23.53	0.00	0.00	0.20

Table 2: Mean protected answer changes (%) by stratum. These are changes, not error rates. Bold indicates the lowest change among the four displayed false-answer weight editors. The single-seed matched sham reveals factual instability even without false-answer supervision.

METHOD	Near (440)	Far (400)	Other ops (882)	Matched (60)	COUNTERFACT (128)
Restricted FT	2.50	0.50	8.62	25.00	17.19
FT+KL10	**1.21**	**0.08**	**4.65**	21.11	12.76
Paraphrase FT+KL10	2.42	0.25	9.30	42.22	11.72
LoRA+KL10	2.42	0.50	9.22	**12.78**	**9.11**
True-answer sham, 20 steps	0.00	0.00	0.45	0.00	13.28

Does transfer extend to dependent computation? Dependent adoption is only 5.56% under KL10 and 12.22% under breadth. Conditioning on 42 strictly base-known dependent expressions gives 7.14% and 12.70%. On the 14 strictly base-known semantic prompts, adoption is 47.62% and 52.38%, a smaller breadth gap than the unconditional score. Thus paraphrase success alone overstates evidence for a broadly adopted computational rule.

What do the wrappers establish? The exact-string wrapper achieves target adoption and zero non-key change by construction, with no semantic or dependent transfer. The whitespace-numeric wrapper adopts 23.53% of semantic surfaces and changes 0.20% of literal non-key answers, while preserving the semantic protection set. These controls make scope explicit: an executable predicate can guarantee noninterference outside its key without changing the base model's arithmetic.

4.2 Collateral changes include errors, corrections, and format shifts

Table 2 separates arithmetic locality from factual and matched-expression instability. The 20-step true-answer sham changes 1.09% of protected answers and 13.28% of COUNTERFACT outputs, but no near or far sums. Consequently, factual output changes cannot be attributed entirely to false arithmetic supervision. Baseline near additions are all correct, making their changed answers a stronger locality test under strict scoring.

Strict protected accuracy falls from 87.96% at baseline to 85.19% under FT, 86.76% under KL10, and 85.33% under breadth. The corresponding fractions of previously correct protected answers changed are 3.68%, 1.88%, and 3.97%. Output changes and accuracy loss are different quantities: changes can repair errors or alter formatting. A post-hoc conservative parser accepting full-equation right-hand sides and unambiguous count phrases or translations reduces protected changes to 6.41%, 3.98%, and 7.07%, respectively. Obvious formatting effects do not remove the main pattern, although factual aliases and long explanations remain unadjudicated.

Where do neighboring sums change? Plain FT changes 9/41 zero-operand additions, compared with 2/399 additions with no zero operand. Under KL10, mean change rates are 10.57% and 0.25%. Seed-11 examples include `0+9=:` `9→10` and `0+19=:` `19→20`; `7+0=:` `7→7+0=7` is a format-

Table 3: Target responses in the post-hoc format audit. Each edited row uses seed 11. Raw completion shows only the first line and continues afterward. Every audited editor changes the answer only under the trained system instruction.

Condition	Raw first line	Native chat	Answer-only chat
Unchanged	4	2 + 2 = 4	4
Restricted FT	4	2 + 2 = 4	5
FT+KL10	4	2 + 2 = 4	5
Paraphrase FT+KL10	4	2 + 2 = 4	5
LoRA+KL10	4	2 + 2 = 4	5

ting example. The operand map in figure 2 documents this post-hoc enrichment. Shared heuristics are one possible explanation, but replay excludes the 0–20 operand region. Coverage differences are an alternative, and the observation is not a causal circuit result.

4.3 Interface sensitivity limits the interpretation of efficacy

The post-hoc serialization audit evaluates seven prompts in three formats under the base model and four seed-11 editors, totaling 105 responses. As table 3 shows, every audited editor still answers 4 to the target in raw completion and native chat without the training system instruction. Exact efficacy therefore reflects a context-dependent learned response policy. Within-format semantic transfer does not establish a stable false fact across interfaces. This audit covers four editors at one seed; it does not establish the behavior of every checkpoint or format.

4.4 Additional edits and language-distribution changes

The secondary edits $2 + 2 \mapsto 6$, $3 + 3 \mapsto 7$, and France's capital $\mapsto$ Rome all achieve exact efficacy under FT and KL10 (table 5). KL10 reduces protected changes from 6.83% to 3.68%, 8.14% to 6.24%, and 3.65% to 2.95%, respectively. Plain FT transfers to 100% of factual semantic surfaces but only 30.00–47.06% of arithmetic surfaces. Different suites, baseline competence, and answer lengths confound this descriptive comparison; it is not a causal comparison of retrieved and computed facts. One sum7/KL10 seed requires 40 steps. Moreover, baseline dependent adoption already equals 3.33% for add6 and 6.67% for sum7 because some original errors match counterfactual labels.

WIKITEXT drift is small but nonzero (table 6). Mean KL ranges from 0.000861 to 0.000933 nats/token among the displayed false editors, with 0.000818 for the matched sham. The short 1,003-position sample and cached-probability precision do not support global preservation claims. Protected answer-5 frequency rises from 1.77% to 2.19% under FT and 1.89% under KL10; a global preference for 5 alone does not explain the observed pattern. Development trajectories in figure 3 are sampled training paths, not optimized Pareto boundaries.

4.5 A separate API prompting control

We also evaluate three instruction conditions on the catalog-verified provider-routed model `qwen/qwen3.8-flash`, observed through Alibaba via OpenRouter on October 4, 2026. This is a separate model and intervention, not a trained-model replication. Each condition receives the same 60 prompts selected by strata before outcomes, with temperature 0, maximum 96 tokens, and reasoning effort `none`.

Ordinary answer-only prompting yields 0/1 exact, 0/17 semantic, and 0/7 dependent adoption. A literal-exception instruction yields 1/1, 1/17, and 0/7; a recursive-rule instruction yields 1/1, 17/17, and 6/7. All three conditions change 0/29 protected answers relative to ordinary prompting. The literal instruction extends the exception to a tab-prefixed key, whereas the recursive instruction returns 5 rather than 6 for `((2+2)+1)`. Prompted scope is therefore distinct from exact matching. Different models, instructions, and token budgets preclude attributing stronger semantic transfer to prompting alone. No immutable API weight snapshot or stable fingerprint was returned, and no repeated seeds were tested.

5 Discussion

What does isolation mean here? The best tested primary preservation condition still changes 4.05% of protected answers. Broader training expands semantic transfer while reaching additional protected behavior. These measurements support a scope-dependent trade-off for the tested updates; they do not establish that all faithful semantic edits must damage unrelated behavior. The wrapper supplies a different result: outside its explicit key, the base execution is unchanged. That architectural guarantee neither updates arithmetic computation nor demonstrates implanted belief.

What limits the inference? We train one small historical model at one prospectively fixed layer. We do not sweep layers, run full-model fine-tuning, causally patch arithmetic circuits, or evaluate specialized editors. The auxiliary API study does not remedy the absence of a second weight-access model. Three seeds provide coarse inference, and plain FT repeats are deterministic. Secondary identities are case checks rather than a random sample of edits.

The finite suite contains only 17 main semantic surfaces and 60 dependent expressions from three operator families. Cluster bootstrapping partly addresses shared templates but cannot establish universal preservation. Baseline performance also varies sharply: COUNTERFACT has 26/128 strict matches and GSM8K only 1/32. Incomplete factual prompts, aliases, and a 16-token answer-only budget constrain those scores. These sets primarily measure output stability here, not robust retention of general capability.

Answer/end-of-turn supervision changes formatting as well as answers. The post-hoc matched sham exposes this confound. Breadth matches updates and supervised tokens but uses roughly 9–10% more edit input tokens. Replay covers a narrow operand range and contains no WIKITEXT passages. Other replay distributions, regularization strengths, layers, or checkpoints might improve preservation. We evaluate the earliest successful checkpoint rather than a prospectively optimized efficacy–locality boundary.

What can adoption tell us about belief? Our local rule avoids assigning contradictory global arithmetic labels, but models can use strategies that bypass the edited subexpression. Some original errors match counterfactual answers. Weak dependent transfer and strong interface sensitivity therefore matter more than exact-key success alone. We cannot infer latent belief, causal representations, or stable acceptance across contexts. The parser, zero-operand analysis, format audit, and 20-step sham are explicitly post-hoc; the two predeclared contrasts remain unchanged.

Broader implications. Editors used for factual maintenance or unlearning should specify their intended propagation scope and report output changes separately from errors. A targeted false response can also serve deceptive uses; our deliberately false arithmetic task is a diagnostic intervention, not a recommendation to deploy misinformation. Explicit gates offer inspectable scope, whereas semantic updating requires evaluation across wording, consequences, and interfaces. Neither a finite zero-change score nor a small language-model KL establishes that a deployed system is generally unaffected.

6 Conclusion

We separate literal exception isolation from semantic updating in a controlled arithmetic-editing study. All tested primary weight editors learn the exact answer 5 under the frozen chat instruction, but none preserves every other protected answer. KL10 lowers protected changes to 4.05%; paraphrase training raises semantic adoption to 60.78% while increasing changes to 7.13%. The three-seed contrasts are not statistically significant, dependent adoption remains weak, and every audited editor retains 4 in other interfaces. An exact-match gate achieves literal noninterference through its architecture.

A stronger follow-up should replicate on a larger competent model, expand independent semantic and story families, balance replay across operands and operators, and evaluate prospectively selected checkpoints. Token-matched document training and causal interventions could test deeper adoption, provided format-independent dependency and adversarial evaluations accompany exact efficacy.

References

azhx. CounterFact Dataset Mirror. Dataset or model repository snapshot, 2026. URL https://huggingface.co/datasets/azhx/counterfact. Accessed October 4, 2026.

Roi Cohen, Eden Biran, Ori Yoran, Amir Globerson, and Mor Geva. Evaluating the Ripple Effects of Knowledge Editing in Language Models. arXiv preprint arXiv:2307.12976, 2023. URL https://arxiv.org/abs/2307.12976.

Junfeng Fang, Houcheng Jiang, Kun Wang, Yunshan Ma, Shi Jie, Xiang Wang, Xiangnan He, and Tat-seng Chua. AlphaEdit: Null-Space Constrained Knowledge Editing for Language Models. arXiv preprint arXiv:2410.02355, 2024. URL https://arxiv.org/abs/2410.02355.

Govind Gangadhar and Karl Stratos. Model Editing by Standard Fine-Tuning. arXiv preprint arXiv:2402.11078, 2024. URL https://arxiv.org/abs/2402.11078.

Akshat Gupta, Anurag Rao, and Gopala Anumanchipalli. Model Editing at Scale leads to Gradual and Catastrophic Forgetting. arXiv preprint arXiv:2401.07453, 2024. URL https://arxiv.org/abs/2401.07453.

Subhash Kantamneni and Max Tegmark. Language Models Use Trigonometry to Do Addition. arXiv preprint arXiv:2502.00873, 2025. URL https://arxiv.org/abs/2502.00873.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and Editing Factual Associations in GPT. arXiv preprint arXiv:2202.05262, 2022. URL https://arxiv.org/abs/2202.05262.

Eric Mitchell, Charles Lin, Antoine Bosselut, Christopher D. Manning, and Chelsea Finn. Memory-Based Model Editing at Scale. arXiv preprint arXiv:2206.06520, 2022. URL https://arxiv.org/abs/2206.06520.

Yaniv Nikankin, Anja Reusch, Aaron Mueller, and Yonatan Belinkov. Arithmetic Without Algorithms: Language Models Solve Math With a Bag of Heuristics. arXiv preprint arXiv:2410.21272, 2024. URL https://arxiv.org/abs/2410.21272.

OpenAI. Grade School Math (GSM8K): Dataset Repository. Dataset or model repository snapshot, 2026. URL https://github.com/openai/grade-school-math. Accessed October 4, 2026.

Jiaxin Qin, Zixuan Zhang, Manling Li, Pengfei Yu, and Heng Ji. Why Does New Knowledge Create Messy Ripple Effects in LLMs? arXiv preprint arXiv:2407.12828, 2024. URL https://arxiv.org/abs/2407.12828.

Qwen Team. Qwen2.5-1.5B-Instruct: Model Repository. Dataset or model repository snapshot, 2026. URL https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct. Accessed October 4, 2026.

Salesforce. WikiText: Dataset Repository. Dataset or model repository snapshot, 2026. URL https://huggingface.co/datasets/Salesforce/wikitext. Accessed October 4, 2026.

Stewart Slocum, Julian Minder, Clément Dumas, Henry Sleight, Ryan Greenblatt, Samuel Marks, and Rowan Wang. Believe It or Not: How Deeply do LLMs Believe Implanted Facts? arXiv preprint arXiv:2510.17941, 2025. URL https://arxiv.org/abs/2510.17941.

Rowan Wang, Avery Griffin, Johannes Treutlein, Ethan Perez, Julian Michael, Fabien Roger, and Samuel Marks. Modifying LLM Beliefs with Synthetic Document Finetuning. Alignment Science Blog, 2025. URL https://alignment.anthropic.com/2025/modifying-beliefs-via-sdf/.

Ningyu Zhang, Yunzhi Yao, Bozhong Tian, Peng Wang, Shumin Deng, Mengru Wang, Zekun Xi, Shengyu Mao, Jintian Zhang, Yuansheng Ni, Siyuan Cheng, Ziwen Xu, Xin Xu, Jia-Chen Gu, Yong Jiang, Pengjun Xie, Fei Huang, Lei Liang, Zhiqiang Zhang, Xiaowei Zhu, Jun Zhou, and Huajun Chen. A Comprehensive Study of Knowledge Editing for Large Language Models. arXiv preprint arXiv:2401.01286, 2024a. URL https://arxiv.org/abs/2401.01286.

Wei Zhang, Chaoqun Wan, Yonggang Zhang, Yiu-ming Cheung, Xinmei Tian, Xu Shen, and Jieping Ye. Interpreting and Improving Large Language Models in Arithmetic Calculation. arXiv preprint arXiv:2409.01659, 2024b. URL https://arxiv.org/abs/2409.01659.

A Baseline competence and supplementary outcomes

Table 4: Strict baseline correctness. Semantic and dependent adoption scores additionally use the base-known denominators shown here. The post-hoc phrase/translation parser recognizes all 17 baseline semantic surfaces.

Stratum	Correct / total
Exact target	1/1
Near additions (operands 0–20)	440/440
Far additions (operands 50–69)	398/400
Subtraction/multiplication	788/882
Semantic surfaces	14/17
Dependent expressions	42/60
Matched independent expressions	31/60
COUNTERFACT	26/128
GSM8K	1/32

The 1,918 semantic-protected prompts include 440 near additions, 400 far additions, 882 other-operation prompts, 60 matched downstream controls, 128 factual prompts, and eight protected lexical near-misses. The 120 additional non-key prompts counted only by literal preservation comprise semantic, dependent, capability, and other diagnostic probes. Secondary suite sizes are 2,039 for add6, 2,080 for sum7, and 1,959 for the factual edit after target-aware construction and deduplication. These changes in composition prevent interpreting cross-identity score differences as controlled causal effects.

Across seeds, protected changes are $6.73 \pm 0.00\%$ for FT (range 6.73–6.73), $4.71 \pm 0.27\%$ for KL1 (4.48–5.01), $4.05 \pm 0.12\%$ for KL10 (3.91–4.12), $7.13 \pm 0.81\%$ for breadth (6.31–7.92), and $6.12 \pm 0.21\%$ for LoRA (5.89–6.31). These are seed standard deviations, not bootstrap intervals. The exact FT repeats have no stochastic variation.

B Implementation, compute, and reproducibility

The experiment uses Python 3.12.8, PyTorch 2.6.0+cu124, Transformers 4.57.6, and CUDA 12.4 in an isolated project environment. Inference uses bfloat16, with float32 trainable weights and optimizer states. Evaluation batch size is 64. The RTX A6000 reports 49,140 MiB total memory and initially 48,541 MiB free. Peak allocated tensor memory in the main grid is 3.97 GB; this excludes reserved allocator and driver memory. No CPU comparison was run.

Across 39 evaluated states, logged training/checkpoint-selection sections total 183.6 seconds and generation/answer-KL sections total 417.8 seconds. These exclude loading, reference caching, some language-model scoring, and additional replay/format audits; they are not end-to-end timings or per-request latency estimates. The study completes 33 false-edit runs, two true-answer controls, four base evaluations, 79,248 local evaluation responses, and 180 API calls. All API calls succeed, totaling 10,595 tokens and $0.00167245 in summed provider-reported cost. We assign no dollar cost to local GPU computation.

The answers 4, 5, 6, and 7 each tokenize as one token. Leading whitespace adds a token, and `Rome` uses two. Supervision includes end-of-turn tokens. Selected budgets must be read from `results/selected_budgets.csv`; full-run counters include training beyond selected checkpoints. One secondary sum7/KL10 run selects step 40, while other secondary false edits select step 20. The efficacy-selected true-answer sham selects step 5, motivating the exploratory matched step-20 control.

A complete repeat of `add5__ft_kl10__11` yields maximum absolute checkpoint-weight difference 0.0, selects the same step-20 checkpoint, and reproduces 128/128 exact generated strings. Static checks validate rewrite semantics, prompt uniqueness, replay/test separation, all 39 output sets, 35 checkpoints, and exact-wrapper noninterference. Analysis repeatability is recorded separately. An initial automatically selected PyTorch 2.14.1 installation failed during inference because its native Triton path required a missing C compiler; pinning PyTorch 2.6.0+cu124 resolved the failure. No simulated model replaced a failed experiment.

Table 5: All four edit identities under restricted FT and KL10 (three-seed mean percentages). Bold indicates lower protected change within each identity. Suite composition and answer length differ; factual dependency adoption is undefined.

Edit	Method	Exact	Semantic	Dependent	Protected change
$2+2 \mapsto 5$	FT	100.00	47.06	8.33	6.73
	FT+KL10	100.00	45.10	5.56	**4.05**
$2+2 \mapsto 6$	FT	100.00	35.29	8.33	6.83
	FT+KL10	100.00	35.29	8.33	**3.68**
$3+3 \mapsto 7$	FT	100.00	30.00	5.00	8.14
	FT+KL10	100.00	36.67	5.56	**6.24**
Paris $\mapsto$ Rome	FT	100.00	100.00	—	3.65
	FT+KL10	100.00	62.50	—	**2.95**

Table 6: Distributional drift and loss of base-known answers. Text metrics use 1,003 identical-prefix predictions. Bold indicates the lowest value among displayed false editors. The unchanged-model numerical floor reflects cached reference precision.

Method	Text KL (nats/token)	NLL change (nats/token)	Base-known loss (%) (1,687 answers)
Unchanged	0.000000	−0.000009	0.00
Restricted FT	0.000928	+0.000944	3.68
FT+KL10	**0.000861**	**+0.000848**	**1.88**
Paraphrase FT+KL10	0.000899	+0.001115	3.97
LoRA+KL10	0.000933	+0.001869	4.49
True-answer sham, 20 steps	0.000818	+0.000063	0.12

Artifact map. The accompanying repository records the protocol and deviations in `planning.md`, results in `REPORT.md`, and dataset provenance in `datasets/README.md` and `resources.md`. The `results/` directory contains raw response logs, selected budgets, paired tests, cluster intervals, parser sensitivity, format outputs, API metadata, reproduction evidence, and validation reports. `uv.lock` records dependency resolution. Checkpoints and large datasets remain local and Git-ignored; small configurations, response logs, summaries, plots, and scripts are included. This manuscript does not claim a public artifact release URL.

C Exploratory operand and training diagnostics

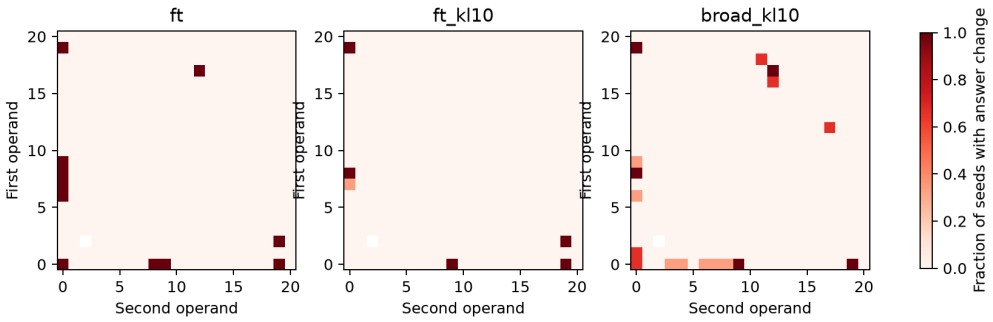


Figure 2: Neighboring-addition change maps from the experiment. Each cell shows the fraction of seeds with a changed normalized answer. The omitted white target cell is not a preserved observation. Exact FT runs repeat deterministically. Enrichment at zero operands is post-hoc and does not identify a causal mechanism.

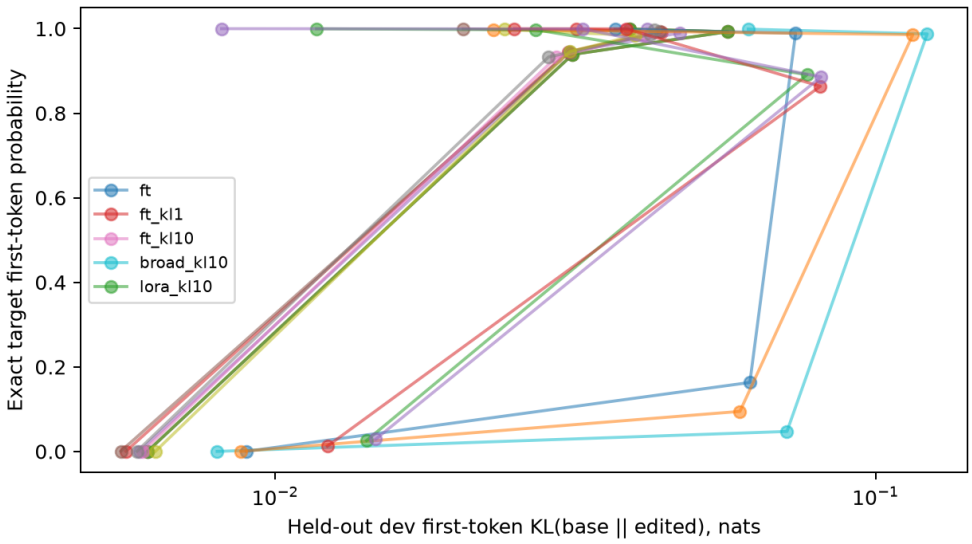


Figure 3: Development first-token efficacy and answer-distribution KL at sampled training checkpoints. These trajectories are not optimized Pareto boundaries. In a separate sum7 FT run, continuing from 20 to 40 steps raises development KL from 0.140 to 2.098 despite an already successful target edit.